Тестовое задание SDET C#

Общее описание

Вам необходимо реализовать отказоустойчивый REST API сервис для управления пользователями на платформе .NET и написать комплексный набор автотестов для проверки его работы. Данные должны храниться в оперативной памяти (in-memory).

Задание разделено на 4 этапа: тест-дизайн, разработка сервиса, валидация и автоматизация тестирования.

Этап 1: Тест-дизайн

Перед тем как переходить к написанию кода, создайте файл test-cases.md в корне проекта.
В нем в текстовом виде (в формате чек-листа) опишите все тест-кейсы (позитивные и негативные), которые необходимы для полной проверки требований к API, описанных ниже.

Этап 2: Реализация REST API сервиса

Создайте проект **ASP.NET Core Web API** (.NET 6 или выше).

2.1. Модели данных и DTO

Создайте класс пользователя **User** (хранится в системе):

- UserId (Guid) — уникальный идентификатор.
- CountryIsoCode (string) — двухбуквенный код страны в верхнем регистре.
- DateCreated (DateTime) — дата создания (выставляется сервером автоматически).
- DateModified (DateTime?) — дата последнего изменения (изначально null).
- Locale (string) — локаль пользователя.

Создайте DTO-класс для запросов **UserDto**:

- Содержит только CountryIsoCode и Locale (идентификатор и даты клиент передавать в теле запроса не должен).

2.2. Интерфейс и реализация репозитория

Создайте интерфейс IUserRepository:

- User Add(User user) — добавляет нового пользователя, возвращает созданный объект.
- bool Update(User user) — обновляет данные существующего пользователя, возвращает true/false.
- User GetById(Guid userId) — возвращает пользователя по ID или null.
- IEnumerable<User> GetAll() — возвращает всех пользователей.

Реализуйте класс InMemoryUserRepository. **Внимание:** учтите требования к работе в многопоточной среде (сервис должен быть потокобезопасным при одновременных запросах).

2.3. Контроллер (UsersController)

Реализуйте следующие эндпоинты по стандартам REST API:

Метод	URL	Описание	Ожидаемый ответ
POST	/api/users	Создание пользователя. Сервер сам генерирует UserId и DateCreated.	201 Created с JSON-телом созданного пользователя.
PUT	/api/users/{userId}	Обновление существующего пользователя. Изменяет	200 OK с обновленными

		CountryIsoCode, Locale и обновляет DateModified.	данными или 404 Not Found.
GET	/api/users/{userId}	Получение пользователя по ID.	200 OK с данными пользователя или 404 Not Found.
GET	/api/users	Получение списка всех пользователей.	200 OK со списком объектов.

Этап 3: Валидация входных данных

Добавьте строгую валидацию для запросов (в контроллере или через DataAnnotations / FluentValidation). При нарушении правил API должен возвращать 400 Bad Request с понятным текстом ошибки:

1. **UserId** (в URL для PUT/GET запросов) — должен быть валидным, не пустым GUID (Guid.Empty запрещен).
2. **CountryIsoCode** — строго 2 буквы латиницы. Проверка должна быть регистронезависимой (при сохранении в память приводить к UPPERCASE).
3. **Locale** — строго формат xx-XX (две строчные буквы латиницы, дефис, две прописные буквы латиницы, например ru-RU, en-US).

Этап 4: Написание автотестов (Основная часть)

Используя любой удобный фреймворк (**xUnit/nUnit/MSTest)** и **Microsoft.AspNetCore.TestHost** (WebApplicationFactory), напишите интеграционные тесты для проверки вашего API.

Требования к автотестам:

1. **Изоляция тестов:** Каждый тест должен запускаться в изолированном окружении. Данные, созданные в Тесте А, не должны влиять на Тест Б (обеспечьте сброс состояния репозитория перед каждым тестом).
2. **Data-Driven тестирование:** Для проверки валидации CountryIsoCode и Locale обязательно используйте параметризованные тесты ([Theory] и [InlineData]). Передайте как минимум по 3-4 невалидных варианта для каждого поля (пустые строки, спецсимволы, цифры, неверная длина).
3. **Проверка схемы и типов:** В позитивных тестах проверяйте не только HTTP-код ответа, но и десериализуйте JSON-ответ. Убедитесь, что все свойства (UserId, DateCreated и т.д.) присутствуют в ответе и имеют корректный формат/значения.

Обязательные тест-сценарии:

- Полноценный **End-to-End сценарий**: Создание через POST -> Получение через GET по ID -> Модификация через PUT -> Проверка изменений через повторный GET.
- Негативные тесты валидации (все типы неверных полей из этапа 3).
- Тест на поиск несуществующего Guid (404 Not Found).

Дополнительные задания (по желанию)

- **Пагинация:** В метод GET /api/users добавить необязательные query-параметры page (номер страницы) и pageSize (размер страницы). Написать автотест, проверяющий корректность разбиения на страницы.
- **Глобальная обработка ошибок:** Реализовать Custom Middleware, которое перехватывает любые непредвиденные Exception в приложении и возвращает клиенту чистый JSON со статусом 500 Internal Server Error. Написать тест, провоцирующий ошибку, для проверки этого Middleware.

- **Логирование в тестах:** Настроить логирование внутри API, а в проекте тестов связать логи приложения с ITestOutputHelper, чтобы логи эндпоинтов выводились в консоль упавшего теста при локальном запуске или в CI.

Требования к сдаче проекта:

- Решение должно быть оформлено в виде архива, содержащего два проекта: UserManagement.API (сервис) и UserManagement.Tests (тесты).
- Все файлы проекта должны быть разложены по смысловым папкам (Models, DTOs, Interfaces, Repositories, Controllers, Tests).
- Все тесты должны успешно проходить по команде dotnet test из корня решения.
- Код должен соответствовать общепринятым правилам Clean Code и C# Conventions (корректные именования, отсутствие "мертвого" кода).

Обрати внимание – результат тестового задания должен быть в формате архива и загружен по ссылке:
https://box.kaspersky.com/u/d/53317deb628f4675bf83/

Выполненное задание обязательно нужно подписать своей фамилией и инициалами, например:
ФамилияИ.О._Тестовое задание_SDETC#.